RX / SPORTS ANALYTICS

OAM Implementation Guide

NBA and NFL opponent-adjusted player models
From neutral history to matchup probabilities

LOOKBACK / NEW SEASONS / CODE LOGIC / PRICING

A guide to the implementation in this repository, including the exact history windows, minimum samples, seasonal carryover, chronological calibration, saved forecast behavior and limitations.

Read section 0 first for the answers to how many games are used and what happens when a new season has little or no history.

The original research PDF is retained separately as a source reference. This manual explains what the code actually implements; advanced proposals in the source papers are not represented as completed features.

Code review date: 20 October 2026
Documentation source: models/Player_Stats/Opponent_Adjusted_Matchup/README.md

Contents

0. Exact history windows and new seasons 3

1. Data and chronological boundaries 4

2. Schedule-adjusted actual versus expected 4

3. Separate opportunity and efficiency features 4

4. Positional matchups and sparse samples 5

5. Applying factors without counting old matchups twice 6

6. Uncertainty and optional lineup scenarios 6

7. Saved markets, odds, Edge and persistence 6

8. Preparation and historical evaluation 7

9. Scope and limitations 7

This package implements an NBA/NFL box-score version of the supplied `Opponent_Adjusted_Matchup_Modeling_NBA_NFL_NCAAF.pdf`. The unchanged source reference and separate `OAM_Implementation_Guide.pdf` accompany the code. This guide describes what is implemented, rather than repeating the source roadmap.

0. Exact history windows and new seasons

Setting	NBA	NFL
Player role history / half-life	20 appearances / 5 games	16 appearances / 3 games
Player and offense efficiency / half-life	60 appearances / 20 games	32 appearances / 8 games
Team workload scenarios	Last 20 games	Last 16 games
Defensive matchup window / half-life	30 allowed games / 8 games	16 allowed games / 4 games
Neutral defensive prior strength k	8 games	10 games
Positional defensive prior strength k	16 games	20 games
Neutral offense prior strength	4 games	6 games
Rolling league prior window	256 eligible feature rows	128 eligible feature rows
Minimum league prior	16 eligible feature rows	16 eligible feature rows
Minimum usable defense games	3	3
Minimum player/team appearances	5	3
Simulations per distribution	12,000	12,000

`config.py` contains these fixed research defaults. Half-lives count appearances in the corresponding series, not calendar days. Each feature requires actual finite observations and positive exposure. A defensive game only counts after the offense/league prior is available, so its effective sample may be smaller than the raw schedule. The first cold-start games initialize neutral priors and are not assigned invented actual/expected labels.

The existing historical repository reads the matchup season and immediately preceding season, including saved regular and postseason games. NBA seasons use starting years with an October boundary; football uses March to keep January/February in the preceding season. No windows reset at opening night. A new season with few games borrows remaining appearances and defensive games from the previous season through the same recency weighting and shrinkage.

If neither season has enough data, estimates remain unavailable. Player identity uses the latest known club across all saved clubs; opportunity ratios then use same-current-club appearances only. A recent trade can therefore leave a player unavailable until the minimum same-club history exists. The shared identity rule rejects player histories older than 450 days. No rookie production or transfer role is fabricated. NCAAF is not enabled in this implementation.

1. Data and chronological boundaries

`historical_data.py` reuses saved NBA Stats/SportsDataverse and nflverse team/ player box scores. Unlike a matchup-only baseline, schedule normalization needs league-wide teams and their actual prior opponents. NFL positional aggregates use the saved player position labels. Missing tracking, routes, man/zone, assignments, injuries, weather and red-zone events are not inferred.

The cutoff is the earlier of snapshot pull time and matchup start. Player and team `event_date` and `available_at` must both be strictly before it. Missing publication times use the existing conservative result-time proxy: eight hours after an explicit start or the following UTC day for date-only records. Revised historical data is not a complete point-in-time publication archive.

Schedule fitting proceeds chronologically within each component. A pending-result heap admits a prior game only when its availability is strictly earlier than the next game's start. Concurrent games do not use one another's final results. All saved input rows are truncated before hashing, fitting or cache lookup.

2. Schedule-adjusted actual versus expected

`features.py` produces aligned actual production A and exposure Q. Expected production E is the prior offense's neutral rate times that game's exposure. Realized exposure is used only to label an already finished defensive game; it removes tempo when estimating efficiency. A future target game's eventual minutes, attempts, carries or targets are never prediction inputs.

For each historical game, `schedule_strength.py` first estimates that defense using results available before its start. It stores this pregame factor. The finished offense's actual production is divided by that factor before entering the offense and league neutral-rate histories:

```
neutral_actual_g = actual_g / pregame_defense_factor_g
neutral_rate = sum(w_g * neutral_actual_g) / sum(w_g * exposure_g)
expected_g = neutral_offense_rate_before_g * observed_exposure_g
```

The offense rate is shrunk toward the previously available league rate using its effective number of games and the configured offense prior strength. If offense history is absent, a sufficiently populated league prior supplies the rate. Without that league warmup, no defensive label is manufactured.

This is a sequential schedule-adjustment estimator, not a jointly solved conference/team regression. Historical factors are frozen as of each prior game; later outcomes never revise those earlier neutralization values.

The defensive factor aggregates allowed counts and expected counts using the same units and exposure basis:

```
w_g = normalized 2^(-appearance_age / defense_half_life)
M_raw = sum(w_g * actual_g) / sum(w_g * expected_g)
n_effective = 1 / sum(w_g^2)
alpha = n_effective / (n_effective + k)
M_shrunk = alpha * M_raw + (1 - alpha)
```

The source example 275+231+270 yards divided by 250+210+300 expected yards gives 776/760 = 1.021053. Its separate shrinkage example M=1.20, effective N=4 and k=6 gives alpha=0.40 and a 1.08 factor. These examples are regression checks; their numbers are not assigned to live teams.

3. Separate opportunity and efficiency features

NBA shot and count opportunities use the same box-score possession estimate `FGA + 0.44*FTA - OREB + TOV` as their denominator. Two-point, three-point and free-throw conversions use their actual attempt denominators. There is no second full pace multiplier: the workload simulation already contains tempo. The 0.44 coefficient is an approximation, not a tracking-derived possession count.

NBA component	Actual / exposure
Two-point attempts, threes attempted, FT attempts	Channel count / estimated possessions
Two-point, three-point and FT conversion	Makes / corresponding attempts
Offensive/defensive rebounds, assists, steals, blocks, turnovers	Channel count / estimated possessions

Rebounds and assists remain box-score opportunity proxies. The model does not claim measured rebound chances, potential assists or full missed-shot event accounting. It does separate attempt-volume effects from shooting conversion.

NFL derives the following conditional components, keeping sacks consistent with the existing recorded-play definition:

NFL component	Actual / exposure
Dropback propensity	(Attempts + sacks) / (attempts + sacks + carries)
Sack propensity	Sacks / dropbacks
Target availability	Targets / passing attempts
Catch efficiency	Receptions / targets
Receiving efficiency	Receiving yards / receptions
Rushing efficiency	Rushing yards / carries
Interception risk	Interceptions / incomplete attempts

Negative net yardage remains observed data. A positive expected denominator is required for a multiplicative label. The model does not relabel per-game yards allowed as a per-carry or per-target defensive efficiency.

4. Positional matchups and sparse samples

NFL RB/WR/TE/QB cohorts are aggregated from actual player rows joined to exact game/team identities. Every recorded member of a cohort must have complete required fields before that cohort contributes. Missing players or unobserved stat families are not replaced with zero. FB/HB labels map to RB.

Catch rate, receiving yards per catch and rushing yards per carry can use a position-specific factor in place of the pooled factor. They are not multiplied by both a full pooled effect and a full position effect. Sparse efficiency cohorts fall back to the team-wide feature. Positional target/carry shares are conditional on team targets/carries, so they affect allocation after team volume has been modeled. Without a usable share factor, allocation remains neutral.

Position estimates use stronger prior strengths than team factors. NBA position splits are not fabricated from absent or unreliable tracking/lineup labels. All fitted factors are bounded to [0.5, 1.8] as an explicit research guardrail, not an optimized setting or a rank-to-percentage conversion. A component without three usable defensive labels stays neutral and is recorded as a fallback.

5. Applying factors without counting old matchups twice

`opponent_adjustment.py` provides an optional context to the shared opportunity engine. Actual source observations remain unchanged and are validated first. The context then neutralizes historical component values using each game's stored pregame factor and applies the upcoming opponent's sampled factor:

```
projected_component = historical_component / historical_factor * upcoming_factor
```

NBA team shot/count pools are adjusted before finite allocation to players and Other. Shooting efficiency is neutralized before Beta posterior fitting; a sampled upcoming conversion factor then changes the posterior probability, bounded to [0,1]. Neutralized successes may be fractional power-likelihood inputs; raw source makes and attempts remain integers.

NFL dropback, sack and target probabilities are changed separately. Positional shares alter target/carry allocation. Catch conversion and yards-per-event efficiency are adjusted separately, preserving the route/target/catch/yards decomposition where the source supports it. Observed target shares remain a proxy for unavailable route participation and targets per route.

The shared engine still enforces finite shot/target/carry pools, regulation 240 team minutes and at most 48 individual NBA minutes. NBA points are generated from separate two-point, three-point and FT makes. PRA and other combinations sum underlying values within the same simulation. NFL QB completions and passing yards share generated receiving totals, rather than independent marginals.

6. Uncertainty and optional lineup scenarios

Upcoming defense factors use an approximate Bayesian bootstrap over the retained allowed/expected records. Exponential random weights multiply recency weights; the same defensive game's weight is shared across feature components. The fixed effective-N shrinkage is applied to each bootstrap ratio. This represents sample uncertainty and component dependence, not a fitted posterior calibration model.

The neutral and adjusted runs share the same simulation seed. Neutral runs still remove historical defense effects but set upcoming factors to 1. The adjusted run samples upcoming defensive strength. The displayed net multiplier is the adjusted simulated mean divided by the neutral simulated mean; it is undefined when the neutral mean is zero. It is an output summary, not a blanket factor applied to the final player statistic.

`lineup_adjustment.py` accepts optional user-supplied scenarios with probabilities, relative component effects and explicit available-at timestamps. Probabilities sum to one per offensive team. A common scenario index controls its component effects so, for example, an active/inactive defensive player is not sampled independently for catch rate and yardage. Scenarios must describe changes from the lineup already implicit in the defensive baseline. There is no automatic injury scrape and no hardcoded effect for an unconfirmed status.

```
[
  {"offense":"CHI","available_at":"2026-10-20T12:00:00Z","probability":0.6,
   "factors":{"nfl:catch:WR":1.02,"nfl:rec_ypc:WR":1.03}},
  {"offense":"CHI","available_at":"2026-10-20T12:00:00Z","probability":0.4,
   "factors":{"nfl:catch:WR":0.99,"nfl:rec_ypc:WR":0.98}}
]
```

These are illustrative scenario inputs, not estimates about any real player. Use exact component names; positional factors replace pooled efficiencies, so a WR-specific scenario should explicitly target the WR component.

7. Saved markets, odds, Edge and persistence

Choose Opponent-Adjusted Matchup in Player Forecast Method for NBA/NFL saved FanDuel games. The table shows OAM probability, OAM fair odds and the corresponding Edge; Apply Criteria uses these selected-model values. WHP remains the default. NCAAF disables OAM while retaining its existing WHP forecasts.

The empirical distribution preserves integer win/loss/push mass. Fair probability is win/(win+loss), excluding refunds; fair odds use that probability. Edge is this probability minus raw market implied probability, in percentage points. EV per dollar is win probability times net payout minus loss probability. The hover shows neutral/adjusted means, net multiplier, dispersion, interval and EV. Diagnostics retain component raw/shrunk means, effective N, sample count, bootstrap spread, positional fallback and latest historical input date.

`prediction_service.py` reads the full saved game once, so changing tabs retains shared simulation states. Forecast and defense-fit identities include eligible league data, cutoff, configuration and implementation hashes. Forecast identities also include saved quote contents and any supplied scenarios. Changed data or code invalidates the relevant cache. Files are atomically saved beneath the ignored `data/modeling/player_stats/opponent_adjusted_matchup/` tree. Market CSVs and credit logs are not changed. Reading or changing methods makes no HTTP calls.

8. Preparation and historical evaluation

```
python -m models.Player_Stats.Opponent_Adjusted_Matchup --sport NBA --prepare --start
    "2026-10-25T23:00:00Z"
python -m models.Player_Stats.Opponent_Adjusted_Matchup --sport NFL --prepare --start
    "2026-10-25T17:00:00Z"
python -m models.Player_Stats.Opponent_Adjusted_Matchup --sport NFL --snapshot
    "data/market_data/your_snapshot.csv" --output "tmp/oam.json"
python -m models.Player_Stats.Opponent_Adjusted_Matchup --sport NFL --backtest-snapshots "archive.json"
     --actuals "actuals.csv" --compare
```

Preparation explicitly loads league-wide player and team data for both seasons. `--refresh` requests source refreshes. `--simulations` changes the simulation count; `--scenarios` reads optional pregame scenarios. No Odds API request is made by preparation or prediction. Dependencies are the existing runtime requirements.

`backtesting.py` requires genuine pregame quote timestamps and actual outcomes keyed by game, player and market. Each forecast is refit under the same cutoff rules as the dashboard. It reports MAE, RMSE, bias, interval coverage, settled and three-outcome Brier scores, three-outcome log loss, empirical CRPS where available and errors by net-multiplier bucket. Target errors use the last prediction per player/game/market; quote diagnostics retain each supplied snapshot/line/side.

`model_comparison.py` compares WHP, UO and OAM on the supplied archive and reports coverage plus metrics on the exact common timestamped quote set. An unavailable model cannot appear better merely by skipping difficult examples. WHP's predictive distribution, not its historical sample, supplies its replay quantiles. There is no parameter search on the evaluation interval and no trained probability calibration map. Reserve a final untouched interval to evaluate incremental benefit. Compilation, math checks and browser checks establish implementation behavior, not forecasting accuracy or profitable edges.

9. Scope and limitations

The implementation covers schedule-adjusted defense, recency, neutral shrinkage, opportunity/efficiency components, observed NFL position cohorts, optional supplied lineup scenarios, simulations and chronological evaluation. The source's tactical interactions, pressure/QB response, slot/corner assignments, tracking chances, weather and NCAAF conference hierarchy need additional data and are not claimed as completed features. Illustrative regression coefficients in the PDF are not used as fitted live coefficients. NFL TD/order props remain unavailable without red-zone opportunities and scorer-order modeling.

Build provenance

The following SHA-256 hashes identify the reviewed calculation/service files. They are documentation provenance, not a runtime cache identity.

models/Player_Stats/Opponent_Adjusted_Matchup/README.md
`78a8fae3cd50e3f49e1d370c82876feeaa945b4187b63f26e7ed529e0fa2a101`

models/Player_Stats/Opponent_Adjusted_Matchup/__init__.py
`d3a207d62867681bbfb9773d5006b52abc8341f09c03352d44f4236c93097a4f`

models/Player_Stats/Opponent_Adjusted_Matchup/__main__.py
`9ed06aa393dc1394ba28cff7ee9fb51170a4c3d4bb632a9b9813bc286430aa86`

models/Player_Stats/Opponent_Adjusted_Matchup/backtesting.py
`1c5007414de8972b38c8b85ba588894e6537258eb4ef3a520600485cde99c666`

models/Player_Stats/Opponent_Adjusted_Matchup/config.py
`adc3804859784116871c2c1665d0e4b8bc7c9cf07d0543ffe35c6e0b737e725f`

models/Player_Stats/Opponent_Adjusted_Matchup/distributions.py
`8ed7b05d77ebd1b35e96146f2df4e372c3bca7538f151d0ef606e7d50b4d1cbe`

models/Player_Stats/Opponent_Adjusted_Matchup/features.py
`872819445982c8c3edfc8f41e3004380f55b7c805f889e2bbcfd9689dde2b71d`

models/Player_Stats/Opponent_Adjusted_Matchup/historical_data.py
`37e9155a9d9787f57da53d56d9017c8da9d1eb6004b8884b3a649bcfa129dfe0`

models/Player_Stats/Opponent_Adjusted_Matchup/lineup_adjustment.py
`36af806dfd5e12dc3608efe1cb2fdc3b3ee5ab9b88c2b7f860cce5036292836b`

models/Player_Stats/Opponent_Adjusted_Matchup/market_comparison.py
`27a9d56a1657d67b83b14e4e8adebcf89a5581075fcb8e3b160c78aaddb5004d`

models/Player_Stats/Opponent_Adjusted_Matchup/model_comparison.py
`60bf30e743005d0dd191e5ae3a24c3919fe01bfd0929ff80a18cc2be538a1f55`

models/Player_Stats/Opponent_Adjusted_Matchup/opponent_adjustment.py
`cc0910be807792fc869bdd75c4e2a2a8b3344730fd81c458fc3dba8ecb8de63e`

models/Player_Stats/Opponent_Adjusted_Matchup/prediction_service.py
`2621ae27bf6a441d26ab688318e67cc359a91fdeab7c45837b6f9397c7776092`

models/Player_Stats/Opponent_Adjusted_Matchup/schedule_strength.py
`4c3c3c8155482aaa3ec2dfc1784c8313e96e6a086622996c08c4fc972d3a3213`

sports_analytics/opponent_adjusted_service.py
`41d55338353c8c06f050068a53fe02e6f029d72d564a648b7b1ef18e24aac779`

models/Player_Stats/Usage_Opportunity/nba_model.py
`a4d1769db11cea448b5131a33495d5c5a23107dcdfb6277f5d4637606e78d91d`

models/Player_Stats/Usage_Opportunity/nfl_model.py
`828b322a5f20e619b5d3ec92c3dbbe62cb1d32bc99cbfe55d4653b291bf14698`

models/Player_Stats/Usage_Opportunity/opportunity_model.py
`116ec170201b0c8aa8f91bdfc2ff1a1151fc1c131be0b5b86cd4f0c673f58553`

models/Player_Stats/Usage_Opportunity/efficiency_model.py
`1fc93ef6b455ec782aa55f4db223fc3f7d96ddbda1e7042d0e0e40e220c075bb`

models/Player_Stats/Usage_Opportunity/distributions.py
`46a0a1b903348080016c37a482e4bfc92ffaf71a6a0ad9e72a8e59207f22a891`

models/Player_Stats/Usage_Opportunity/historical_data.py
`4a5d2b774f95c02ef8e3c3058cf7b7344103fec31e3533ad56c36656ef0dc417`